

Speech Generation, Codecs, Vocoders, and Sequence-to-Audio Models

Mathematics of Deep Learning — Chapter 20

Yin Yang

Foundation Books Project

Roadmap

Speech vs Text Generation

The Pipeline

Phonemes, Durations, Prosody

Acoustic Representations

Alignment

Acoustic Models

Vocoders

Neural Codecs and VQ

Codec Language Models

Speaker, Style, Control

Evaluation and Failure Modes

Summary

Speech vs Text Generation

Running Example: One Short Sentence

Text prompt to speak:

`u = the small robot found a red key.`

At sampling rate $r = 24,000$ Hz and duration $D = 2$ s, the waveform has

$$T = rD = 48,000 \text{ samples.}$$

- Eight content units: $u_{1:8}$ including `<end>`.
- Output is two orders of magnitude longer than the input.
- Speaker s and style c are extra conditioning variables.

Speech generation is long-output modeling even for short text.

Why Speech Is Different from Text

Question	Text generation	Speech generation
Tokens per sentence	~ 10 subwords	$\sim 48,000$ samples
Output is discrete?	yes	continuous waveform
One-to-many?	low	high (voice, rate, pitch)
Latency budget	per token	per second of audio
Failure modes	wrong tokens	artifacts, drift, skips

Conditional speech generator:

$$p_{\theta}(x_{1:T} \mid u_{1:M}, s, c).$$

The same text has many valid spoken outputs; the loss must allow acoustic variation while fixing the content.

Speech-Generation Factorization

Introduce alignment $a_{1:N}$ and acoustic features Y :

$$p(x \mid u, s, c) = \sum_a \int \underbrace{p_\omega(x \mid Y, s, c)}_{\text{vocoder}} \underbrace{p_\eta(Y \mid a, u, s, c)}_{\text{acoustic model}} \underbrace{p_\psi(a \mid u, s, c)}_{\text{timing}} dY.$$

- Timing model decides *when* each token is heard.
- Acoustic model decides *what it sounds like* as frames.
- Vocoder renders frames as waveform samples.

Breaking one hard problem into three smaller conditional problems is the master pattern for this chapter.

The Pipeline

From Text to Waveform: Build It Up

text u

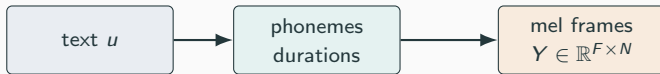
(1) Written text is normalized.

From Text to Waveform: Build It Up



- (1) Written text is normalized.
- (2) Phonemes + durations specify content and timing.

From Text to Waveform: Build It Up



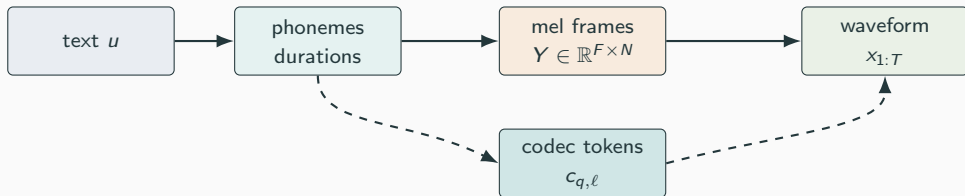
- (1) Written text is normalized.
- (2) Phonemes + durations specify content and timing.
- (3) Acoustic model predicts mel-spectrogram frames.

From Text to Waveform: Build It Up



- (1) Written text is normalized.
- (2) Phonemes + durations specify content and timing.
- (3) Acoustic model predicts mel-spectrogram frames.
- (4) Vocoder reconstructs waveform samples.

From Text to Waveform: Build It Up



- (1) Written text is normalized.
- (2) Phonemes + durations specify content and timing.
- (3) Acoustic model predicts mel-spectrogram frames.
- (4) Vocoder reconstructs waveform samples.
- (5) Alternative path: discrete codec tokens replace Y .

Every later section elaborates one arrow in this chain.

Phonemes, Durations, Prosody

Text Normalization and Phonemes

Written input is not always the best model input.

- Dr. may be spoken as doctor.
- 2026 may be twenty twenty-six or two thousand twenty-six.
- **Text normalization** converts written text to speakable form.
- **Phoneme** = abstract pronunciation unit (sound, not letter).

Tokens for the running example:

$$u_{1:8} = (\text{the, small, robot, found, a, red, key, <end>}).$$

The choice of unit (character / subword / phoneme) trades data efficiency against pronunciation control.

Durations: Token-to-Frame Counts

For $M = 8$ tokens and $N = 24$ acoustic frames, a duration vector $d \in \mathbb{N}_0^M$ with $\sum_m d_m = N$:

$$d = (2, 3, 5, 4, 1, 3, 5, 1), \quad \sum_m d_m = 24.$$

Emphasize red ($3 \rightarrow 5$), shorten robot ($5 \rightarrow 3$):

$$d' = (2, 3, 3, 4, 1, 5, 5, 1), \quad \sum_m d'_m = 24.$$

Token	Original	Emphasized	Change
robot	5	3	-2
red	3	5	+2
Total N	24	24	0

Fixed-length duration edits are zero-sum: time added somewhere must be removed elsewhere.

Duration Loss

A duration model predicts a distribution $q_\eta(d_m \mid u, m)$. Teacher-forced loss against gold durations d_m^* :

$$\mathcal{L}_{\text{dur}}(\eta) = - \sum_{m=1}^M \log q_\eta(d_m^* \mid u_{1:M}, m).$$

Two-headed system: *acoustic* model predicts how frames sound, *duration* model predicts how many frames each token gets.

- Wrong durations $\Rightarrow$ skipped red, repeated key, unnatural pace.
- Right durations $\Rightarrow$ frame-level errors degrade naturalness but not content.

Timing errors are content errors — not vocoder errors — even if the spectrogram is locally accurate.

Acoustic Representations

From Waveform to Mel-Spectrogram

Hop size h samples per frame gives

$$N \approx T/h.$$

For $T = 48,000$, $h = 300$: $N \approx 160$ frames; with $F = 80$ mel bins,

$$Y \in \mathbb{R}^{80 \times 160}, \quad FN = 12,800 \ll 48,000.$$

Hop h	Frames N	Mel values FN
120	400	32,000
240	200	16,000
300	160	12,800
480	100	8,000

Mel features trade waveform detail for a much shorter sequence — but they discard explicit phase.

Representation Sizes for the Running Example

Object	Symbol	Size	Role
Text groups	$u_{1:M}$	$M = 8$	content
Durations	$d_{1:M}$	length 8	timing
Mel features	Y	80×160	acoustic target
Waveform	$x_{1:T}$	$T = 48,000$	final audio
Codec tokens	$c_{q,\ell}$	$Q \times L$	discrete tokens

- Each row names a separate modeling target.
- Compression ratio: $48,000/12,800 \approx 3.75$ from waveform to mel.
- Codec tokens compress further into discrete indices.

Modeling question: which representation does which model predict?

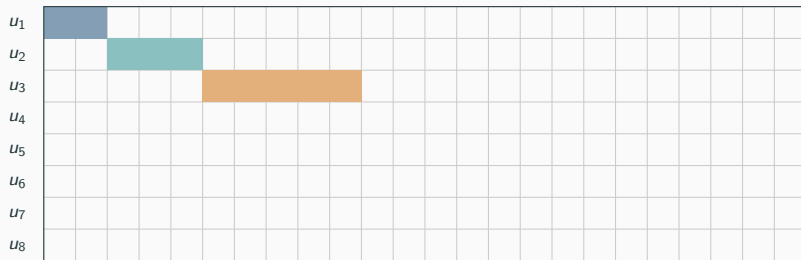
Alignment

Frame-to-Token Alignment, Built Up

u_1																			
u_2																			
u_3																			
u_4																			
u_5																			
u_6																			
u_7																			
u_8																			

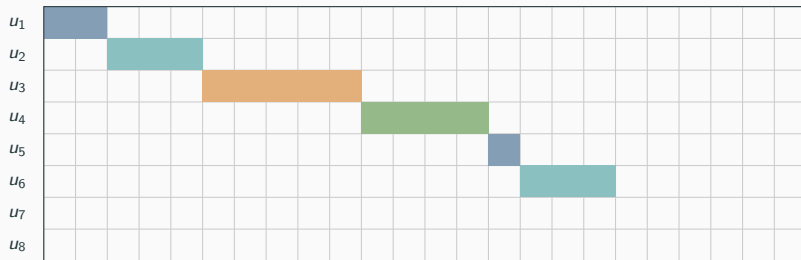
(1) Grid: rows are tokens, columns are frames.

Frame-to-Token Alignment, Built Up



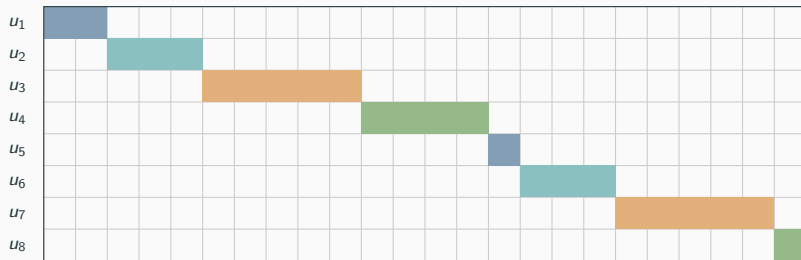
- (1) Grid: rows are tokens, columns are frames.
- (2) First three tokens consume frames 1–10.

Frame-to-Token Alignment, Built Up



- (1) Grid: rows are tokens, columns are frames.
- (2) First three tokens consume frames 1–10.
- (3) Middle tokens fill 11–18 (a just 1 frame).

Frame-to-Token Alignment, Built Up



- (1) Grid: rows are tokens, columns are frames.
- (2) First three tokens consume frames 1–10.
- (3) Middle tokens fill 11–18 (a just 1 frame).
- (4) Path ends: monotone, $a_{n+1} \geq a_n$, no skip.

Speech generation reuses left-to-right monotone alignment — attention failures often start as alignment failures.

Monotone Alignment Search

Score $S_{m,n}$ for assigning token m to frame n . Solve

$$\max_{a_{1:N}} \sum_{n=1}^N S_{a_n,n} \quad \text{s.t.} \quad 1 \leq a_1 \leq \dots \leq a_N \leq M.$$

Dynamic program with $F(m, n)$ = best score ending at (m, n) :

$$F(m, n) = S_{m,n} + \max\{F(m, n-1), F(m-1, n-1)\}.$$

- “Stay” move: $F(m, n-1)$ keeps the same token.
- “Advance” move: $F(m-1, n-1)$ moves to the next token.
- No “go back”: enforces monotonicity by construction.

This is the mathematical core behind FastSpeech-style length regulators and Grad-TTS monotonic alignment search.

Acoustic Models

Acoustic Model: Text $\rightarrow$ Mel

Acoustic model is the conditional

$$p_{\theta}(Y \mid u_{1:M}, s, c), \quad Y \in \mathbb{R}^{F \times N}.$$

Mean absolute error loss:

$$\mathcal{L}_{\text{mel}}(\theta) = \frac{1}{FN} \sum_{f=1}^F \sum_{n=1}^N |Y_{f,n} - \hat{Y}_{f,n}|.$$

Tiny patch example: Y vs $\hat{Y}$,

$$Y = \begin{bmatrix} 1 & 2 & 4 \\ 2 & 2 & 3 \end{bmatrix}, \quad \hat{Y} = \begin{bmatrix} 1 & 3 & 2 \\ 2 & 1 & 5 \end{bmatrix} \Rightarrow \text{abs errors } 0, 1, 2, 0, 1, 2; \text{ MAE} = 1.$$

A low mel loss is necessary but not sufficient — two patches with the same MAE can still sound very different.

Autoregressive vs Non-Autoregressive

Autoregressive (Tacotron-style):

$$p_{\theta}(Y \mid u) = \prod_{n=1}^N p_{\theta}(Y_{:,n} \mid Y_{:,<n}, u).$$

- Each frame conditions on previous predictions.
- Inference is sequential; exposure bias causes drift.

Non-autoregressive (FastSpeech-style):

$$Y = \text{Decoder}(\text{LengthRegulator}(u, d)).$$

- All frames in parallel after duration expansion.
- Stable speaking rate and faster inference.

Auto-regressive buys expressive coupling; non-autoregressive buys speed and explicit duration control.

Vocoders

Three Vocoder Families

Vocoder: $p_{\phi}(x_{1:T} \mid Y)$. Three training signals:

Family		Order	Strength		Risk
Autoregressive (WaveNet)		sample by sample	strong model	likelihood	slow inference
GAN (HiFi-GAN)		parallel	fast synthesis		artifacts, training instability
Diffusion fWave)	(Dif-	iterative denoising	quality / speed knob		many refinement steps

Autoregressive likelihood:

$$\mathcal{L}_{\text{AR}}(\phi) = - \sum_{t=1}^T \log p_{\phi}(x_t \mid x_{<t}, Y).$$

Vocoder choice is an engineering decision driven by quality budget, latency, and artifact tolerance.

GAN Vocoder Losses

Generator G_ϕ , discriminator D_ψ , generated audio $\hat{x} = G_\phi(Y)$.

$$\mathcal{L}_D(\psi) = \mathbb{E}_{x,Y}[(D_\psi(x, Y) - 1)^2] + \mathbb{E}_Y[D_\psi(G_\phi(Y), Y)^2].$$

$$\mathcal{L}_{\text{adv}}(\phi) = \mathbb{E}_Y[(D_\psi(G_\phi(Y), Y) - 1)^2].$$

Auxiliary losses anchor the generator to the conditioning Y :

$$\mathcal{L}_G = \mathcal{L}_{\text{adv}} + \lambda_{\text{fm}}\mathcal{L}_{\text{feature-match}} + \lambda_{\text{mel}}\mathcal{L}_{\text{mel}}.$$

Multi-resolution STFT loss:

$$\mathcal{L}_{\text{MR-STFT}} = \sum_{r \in \mathcal{R}} \left| \left| \text{STFT}_r(\hat{x}) \right| - \left| \text{STFT}_r(x) \right| \right|_1.$$

The adversarial game must be tethered to spectral truth — realism alone allows text-inconsistent audio.

Forward noising:

$$x_k = \sqrt{\bar{\alpha}_k} x_0 + \sqrt{1 - \bar{\alpha}_k} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Noise-prediction objective:

$$\mathcal{L}_{\text{diff}}(\phi) = \mathbb{E}_{x, \epsilon, k} [\|\epsilon - \epsilon_\phi(x_k, k, Y)\|_2^2].$$

Optimal predictor (Bayes-optimal):

$$\epsilon_\phi^*(x_k, k, Y) = \mathbb{E}[\epsilon \mid x_k, k, Y].$$

- Y stays fixed as conditioning; x_k is the noisy waveform.
- Sampling: many denoising steps from large k down to 0.

The MSE target is not arbitrary: it is the conditional mean of the noise still in x_k .

Real-Time Factor

Deployment metric:

$$\text{RTF} = \frac{\text{synthesis time}}{\text{audio duration}}.$$

Setting	Synth time (s)	Audio (s)	RTF
Fast vocoder	0.5	2.0	0.25
Real-time boundary	2.0	2.0	1.00
High-quality slow	3.0	2.0	1.50

$\text{RTF} < 1$ = faster than real time. For an interactive chat assistant, RTF matters as much as MOS.

Quality and latency are independent axes — the best model on paper may be the wrong shipping choice.

Neural Codecs and VQ

Neural Audio Codec

Encoder, quantizer, decoder:

$$z = E_\psi(x), \quad q = Q(z), \quad \hat{x} = D_\omega(q).$$

Vector quantization with codebook $e_1, \dots, e_K \in \mathbb{R}^d$:

$$k^* = \arg \min_{1 \leq k \leq K} \|z - e_k\|_2^2.$$

Example: $z = (1, 2)$, $e_1 = (0, 0)$, $e_2 = (2, 2)$:

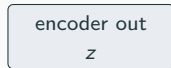
$$\|z - e_1\|_2^2 = 5, \quad \|z - e_2\|_2^2 = 1 \Rightarrow k^* = 2.$$

Training term:

$$\mathcal{L}_{\text{VQ}} = \|\text{sg}(z) - e_{k^*}\|_2^2 + \beta \|z - \text{sg}(e_{k^*})\|_2^2.$$

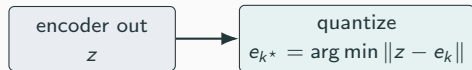
One term moves the code toward the encoder; the other commits the encoder to the chosen code.

Straight-Through VQ Gradient, Step by Step



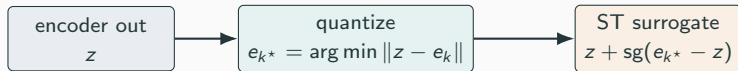
(1) Continuous latent z leaves the encoder.

Straight-Through VQ Gradient, Step by Step



- (1) Continuous latent z leaves the encoder.
- (2) Nearest-code lookup picks e_{k^*} (discrete).

Straight-Through VQ Gradient, Step by Step



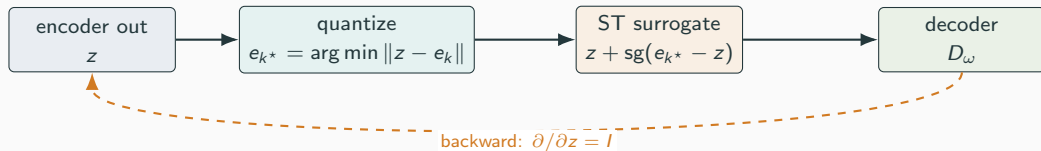
- (1) Continuous latent z leaves the encoder.
- (2) Nearest-code lookup picks e_{k^*} (discrete).
- (3) Forward value = e_{k^*} ; backward derivative = I .

Straight-Through VQ Gradient, Step by Step



- (1) Continuous latent z leaves the encoder.
- (2) Nearest-code lookup picks e_{k^*} (discrete).
- (3) Forward value $= e_{k^*}$; backward derivative $= I$.
- (4) Decoder reconstructs $\hat{x}$ from the code.

Straight-Through VQ Gradient, Step by Step



- (1) Continuous latent z leaves the encoder.
- (2) Nearest-code lookup picks e_{k^*} (discrete).
- (3) Forward value = e_{k^*} ; backward derivative = I .
- (4) Decoder reconstructs $\hat{x}$ from the code.
- (5) Gradients flow back *around* the discrete step.

Discrete forward, continuous backward — enough to train an encoder for a non-differentiable code lookup.

Residual VQ and Bitrate

Approximate z_t by a sum of code vectors from Q codebooks:

$$z_t \approx e_{c_{1,t}}^{(1)} + e_{c_{2,t}}^{(2)} + \cdots + e_{c_{Q,t}}^{(Q)}.$$

Each stage minimizes the residual; with a zero vector in the codebook, residual norm is monotonically non-increasing.

Raw token bitrate:

$$B = Q f_z \log_2 K \quad \text{bits per second.}$$

Q	K	f_z	B
3	16	50	600 bps
4	1024	75	3.0 kbps
8	1024	75	6.0 kbps

Codec tokens compress audio enough that sequence models can generate seconds, not samples.

Codec Language Models

Codec Tokens as a Sequence Modeling Problem

Token matrix $C \in \{1, \dots, K\}^{Q \times L}$. First-codebook autoregressive model:

$$p_{\theta}(c_{1:L}^{(1)} \mid u, p) = \prod_{\ell=1}^L p_{\theta}(c_{\ell}^{(1)} \mid c_{<\ell}^{(1)}, u, p).$$

Full multi-codebook factorization:

$$p_{\theta}(C \mid u, p) = \prod_{\ell=1}^L \prod_{q=1}^Q p_{\theta}(c_{q,\ell} \mid C_{<(\ell,q)}, u, p).$$

- u = text tokens; p = acoustic prompt for voice cloning.
- Coarse codebook first (structure), residual codebooks later (detail).

Voice cloning falls out of conditioning on a short audio prompt — no new text content, just acoustic context.

Codec-Token Loss and Counting

Cross-entropy summed over time and codebook:

$$\mathcal{L}(\theta) = - \sum_{\ell=1}^L \sum_{q=1}^Q \log p_{\theta}(c_{q,\ell} \mid C_{<(\ell,q)}, u, p).$$

Counting example: $Q = 3$ codebooks, $L = 4$ frames give

$$QL = 12 \text{ prediction terms.}$$

If the mean per-term loss is 1.2, total = $12 \cdot 1.2 = 14.4$.

Decoding tradeoff:

- Low temperature: stable but flat.
- High temperature: expressive, but clicks, drift, wrong words.

Same decoding math as text LMs — but here failures are audible artifacts, not silent typos.

Speaker, Style, Control

Conditioning Variables

Full model:

$$p_{\theta}(x_{1:T} \mid u_{1:M}, s, c).$$

Three orthogonal pressures on the waveform:

- $u_{1:M}$ — the *words* to say.
- $s \in \mathbb{R}^d$ — speaker embedding (*identity*).
- c — style: rate, pitch range, emotion, language, channel, reference audio.

Diagnostic principle: change one input, check the others.

- Speed control changes pronunciation? Rate and content are entangled.
- Speaker prompt changes the words? Acoustic context overrides text.
- Emotion destroys speaker similarity? Identity and style are entangled.

Controllability is measured by what stays fixed when one conditioning variable changes.

Safety of Voice Conditioning

Voice identity is personal information.

- Speaker embeddings + prompt-based cloning enable imitation from seconds of reference audio.
- Modeling choice and safety are not separable: the conditioning determines what the system is *capable* of imitating.

Operational expectations:

- Consent for any voice that is cloned.
- Logging of which voices were used and by whom.
- Misuse checks before high-fidelity zero-shot models ship.

Voice is a personal identifier — treat conditioning variables with the same care as biometric data.

Evaluation and Failure Modes

Four Axes, Four Checks

No single number evaluates a speech generator.

Question	Check	What it can miss
Sounds natural?	MOS, listening tests	word accuracy, voice
Says the words?	ASR WER, human transcript	naturalness, emotion
Right speaker?	similarity rating or embedding	content errors
Deployable?	RTF, latency, memory	perceptual quality
Robust?	stress tests on text and prompts	average quality

Pick a basket of metrics that, together, would catch the failures you have actually seen.

Failure Triage by Subsystem

System	Symptom	Likely subsystem	Next diagnostic
Toy A	repeats red key	duration / alignment	inspect frame-to-token path
Toy B	correct words, buzzy audio	vocoder / codec decoder	compare mel target and reconstruction
Toy C	right words, wrong voice	speaker / style condition	speaker similarity by prompt group

Failure-to-cause table (mental model):

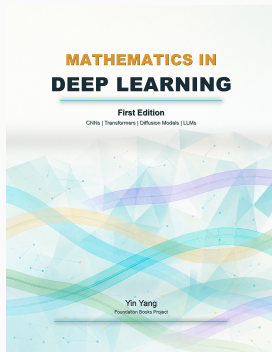
- Skipped / repeated words → alignment, duration.
- Buzz, metallic, rough consonants → vocoder, codec.
- Wrong identity → conditioning s , prompt p .
- Flat rhythm → prosody modeling.
- Slow synthesis → architecture, hardware mismatch.

Route debugging by symptom, not by hunch: the subsystem boundary is the loss boundary.

Summary

- **Long output:** 24,000 samples per second of audio forces representation choices.
- **Factorization:** timing $\rightarrow$ acoustic $\rightarrow$ waveform, with codec tokens as an alternative discrete path.
- **Durations:** $\sum_m d_m = N$ couples token and frame grids; edits are zero-sum.
- **Acoustic models:** mel-MAE is necessary, not sufficient; AR trades speed for expressivity.
- **Vocoders:** AR / GAN / diffusion solve the same conditional generation problem with different signals.
- **Codecs:** VQ + straight-through gradient turn waveform generation into discrete sequence modeling.
- **Codec LMs:** prompt-conditioned voice cloning, but decoding errors become audible.
- **Evaluation:** naturalness, intelligibility, similarity, latency, robustness — four to five axes, not one.

Next: music, environmental sound, and general audio generation share the same scaffolding — different content, same chain of conditional representations.



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/mathematics-in-deep-learning
Code	github.com/foundation-books/mathematics-in-deep-learning-code
Paperback	amazon.com/dp/BOH1ZZHDT2
Hardcover	amazon.com/dp/BOH1ZL11MJ
Kindle	amazon.com/dp/B0GX32M8SP
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.